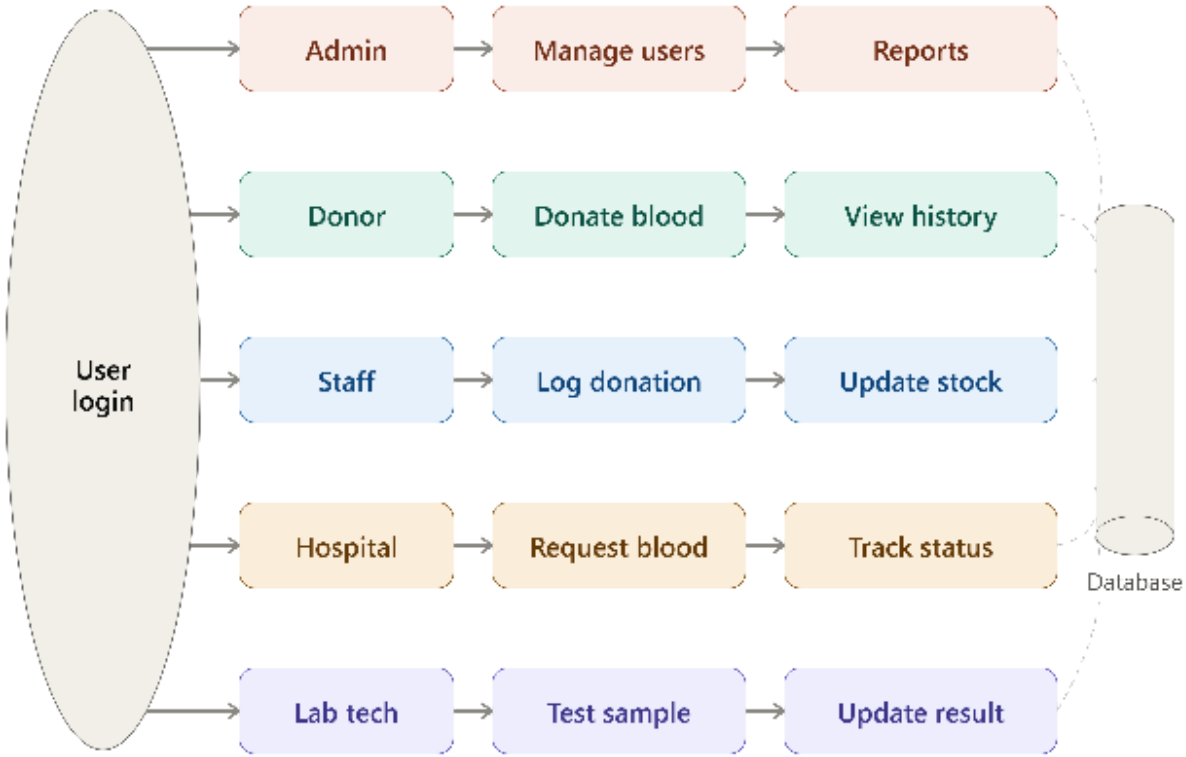
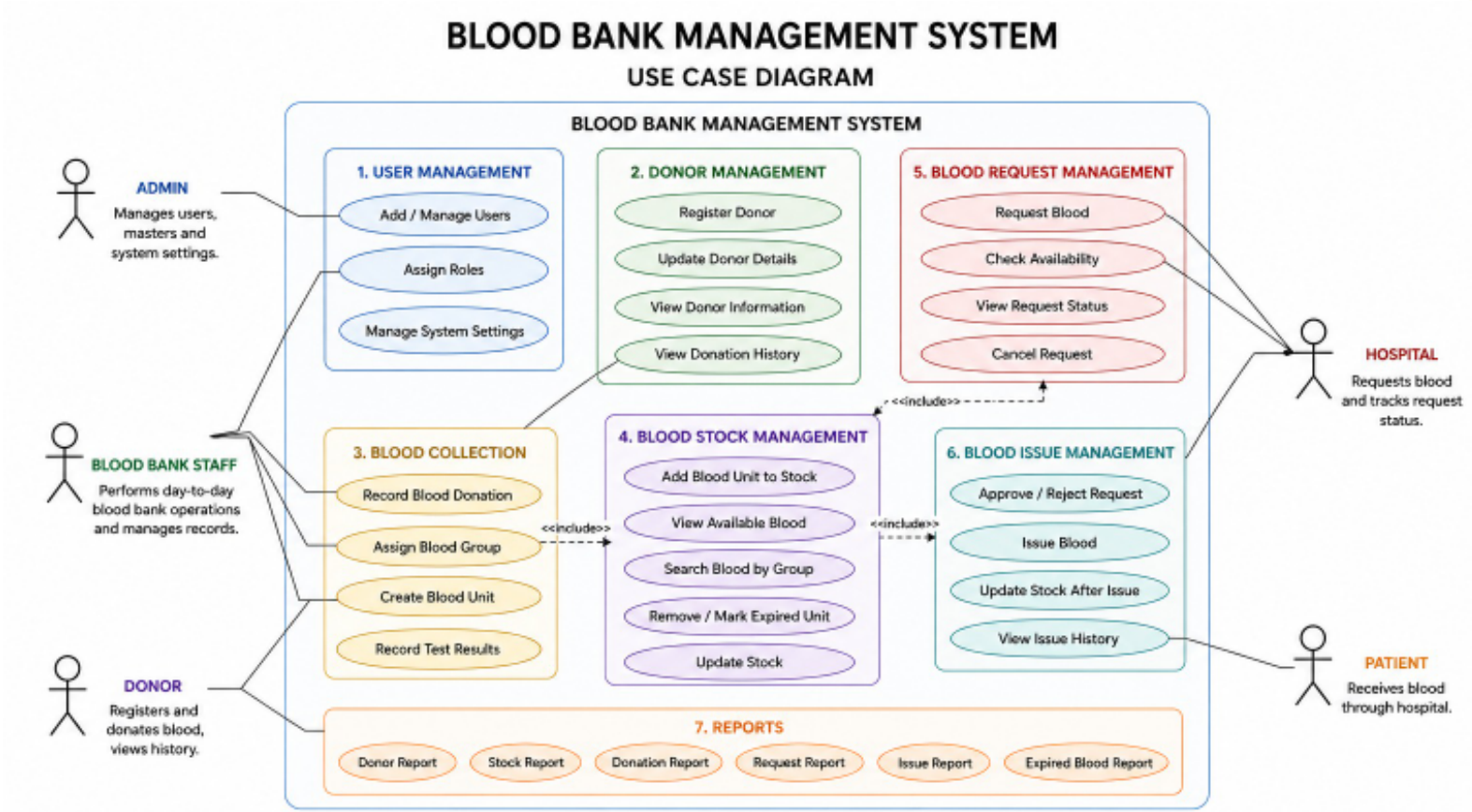
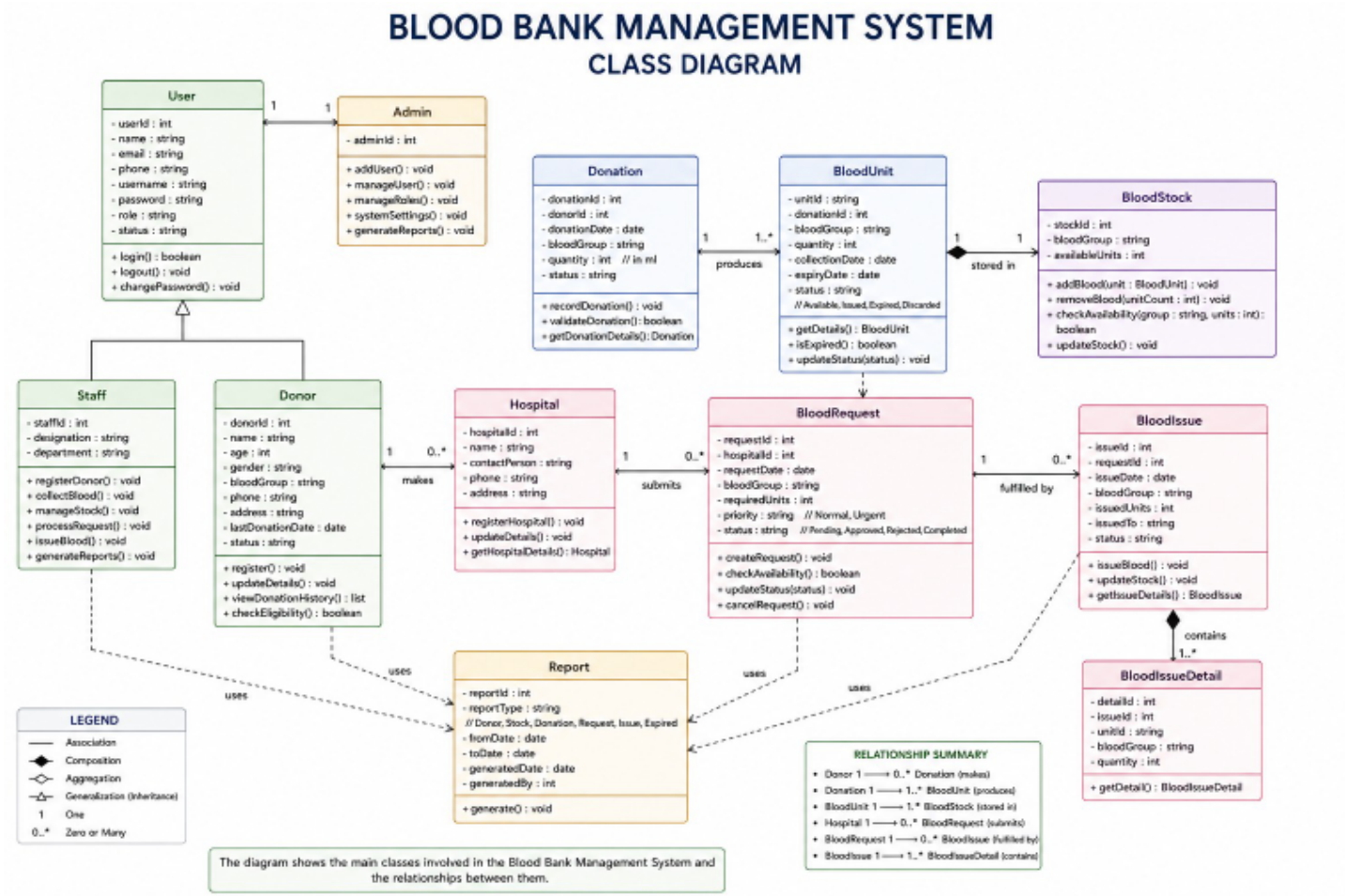
SYSTEM ARCHITECTURE

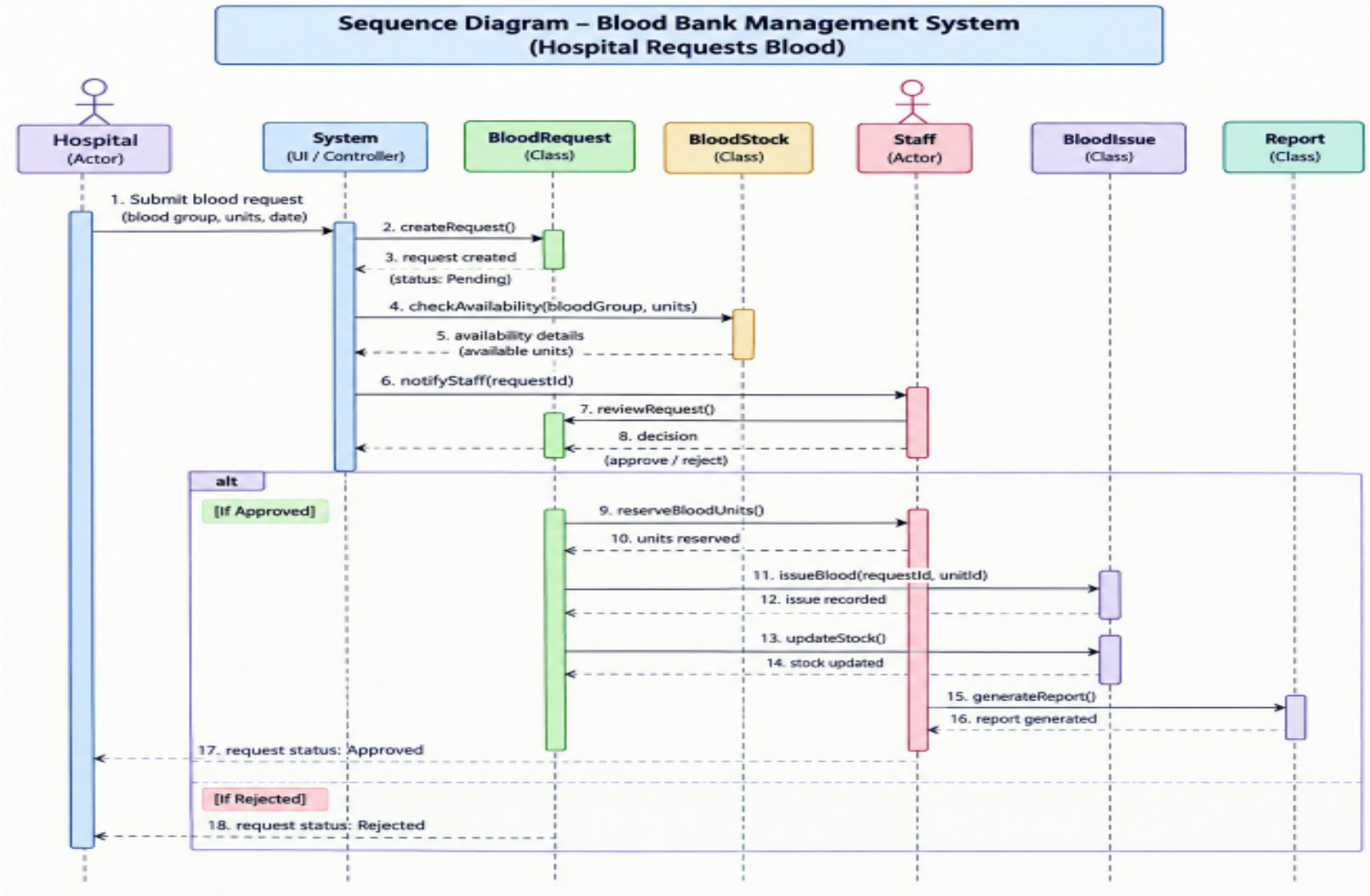


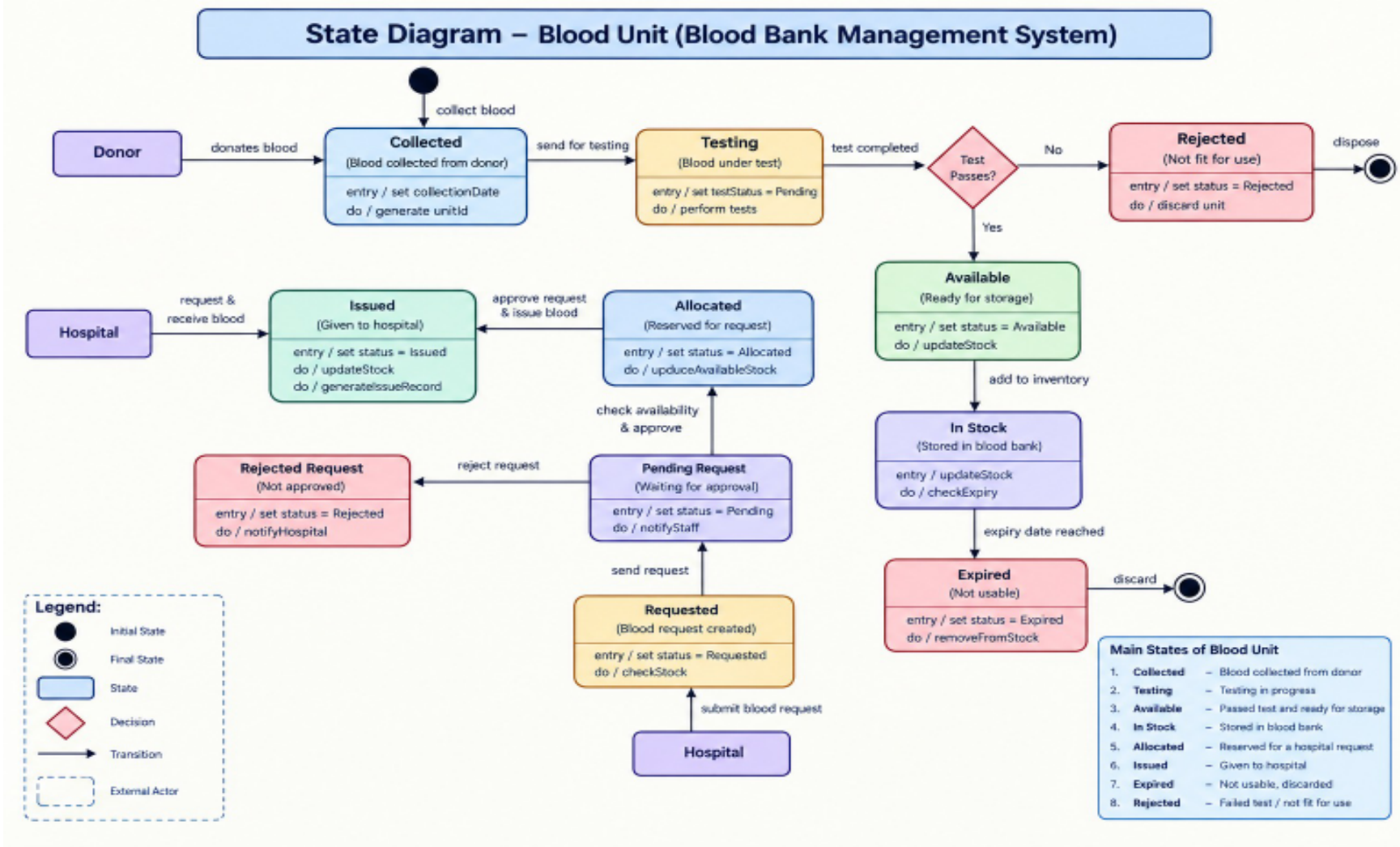
USECASE DIAGRAM

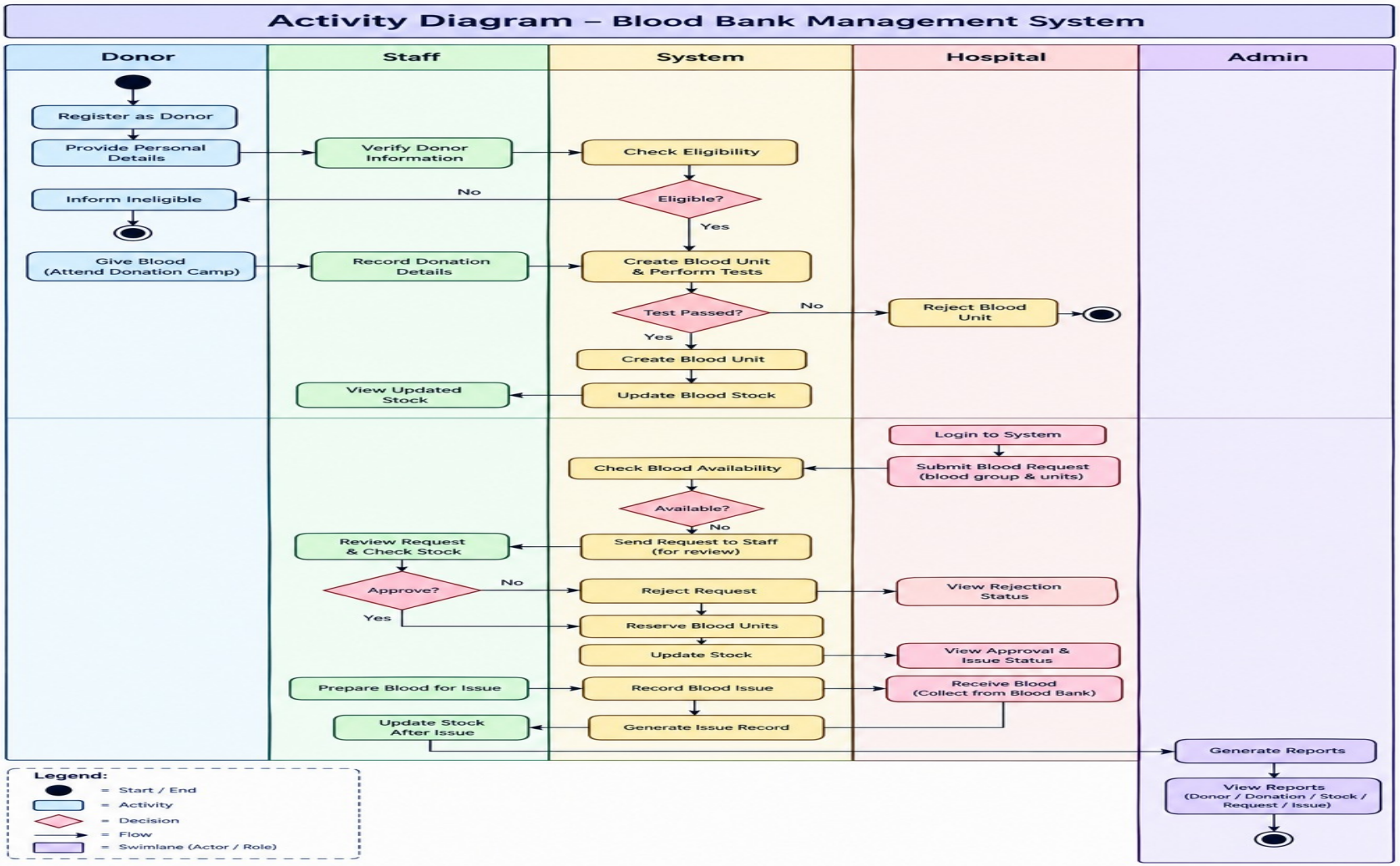


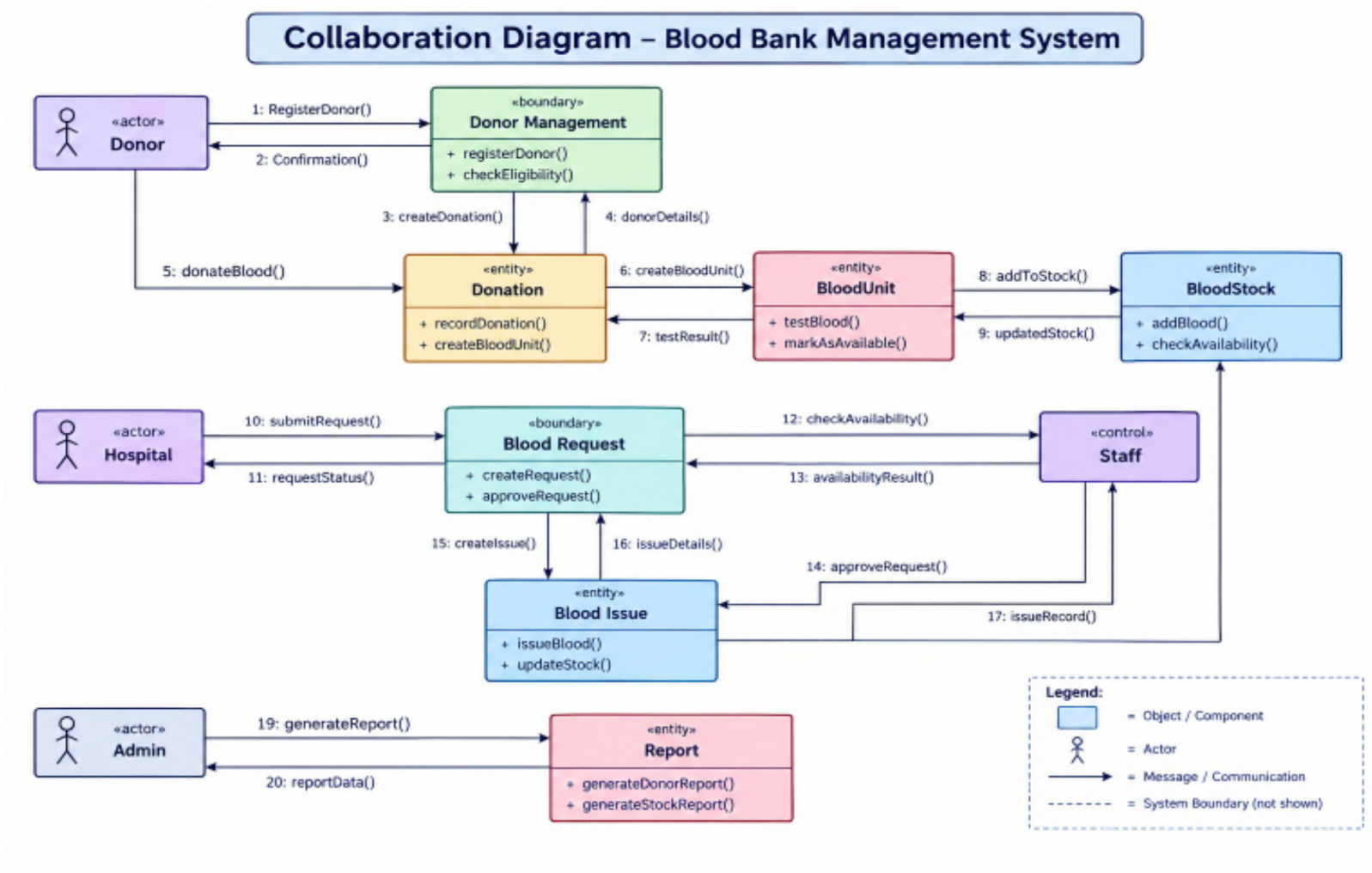
CLASS DIAGRAM

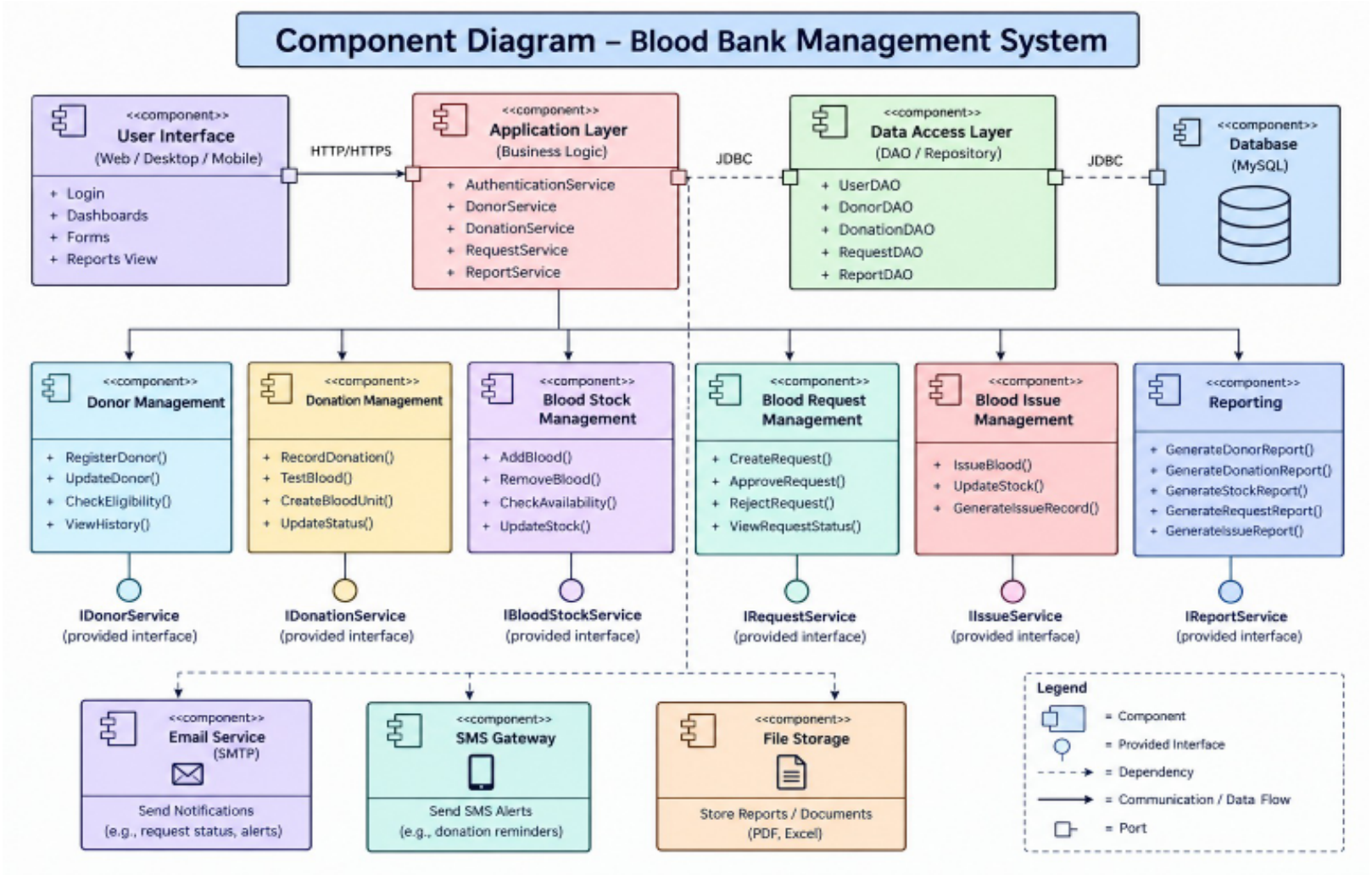



Sequence Diagram – Blood Bank Management System
(Hospital Requests Blood)
Hospital
(Actor)
System
(UI / Controller)
BloodRequest
(Class)
BloodStock
(Class)
Staff
(Actor)
BloodIssue
(Class)
Report
(Class)
1. Submit blood request
(blood group, units, date)
2. createRequest()
3. request created
(status: Pending)
4. checkAvailability(bloodGroup, units)
5. availability details
(available units)
6. notifyStaff(requestId)
7. reviewRequest()
8. decision
(approve / reject)
alt
[If Approved]
9. reserveBloodUnits()
10. units reserved
11. issueBlood(requestId, unitId)
12. issue recorded
13. updateStock()
14. stock updated
15. generateReport()
16. report generated
17. request status: Approved
[If Rejected]
18. request status: Rejected

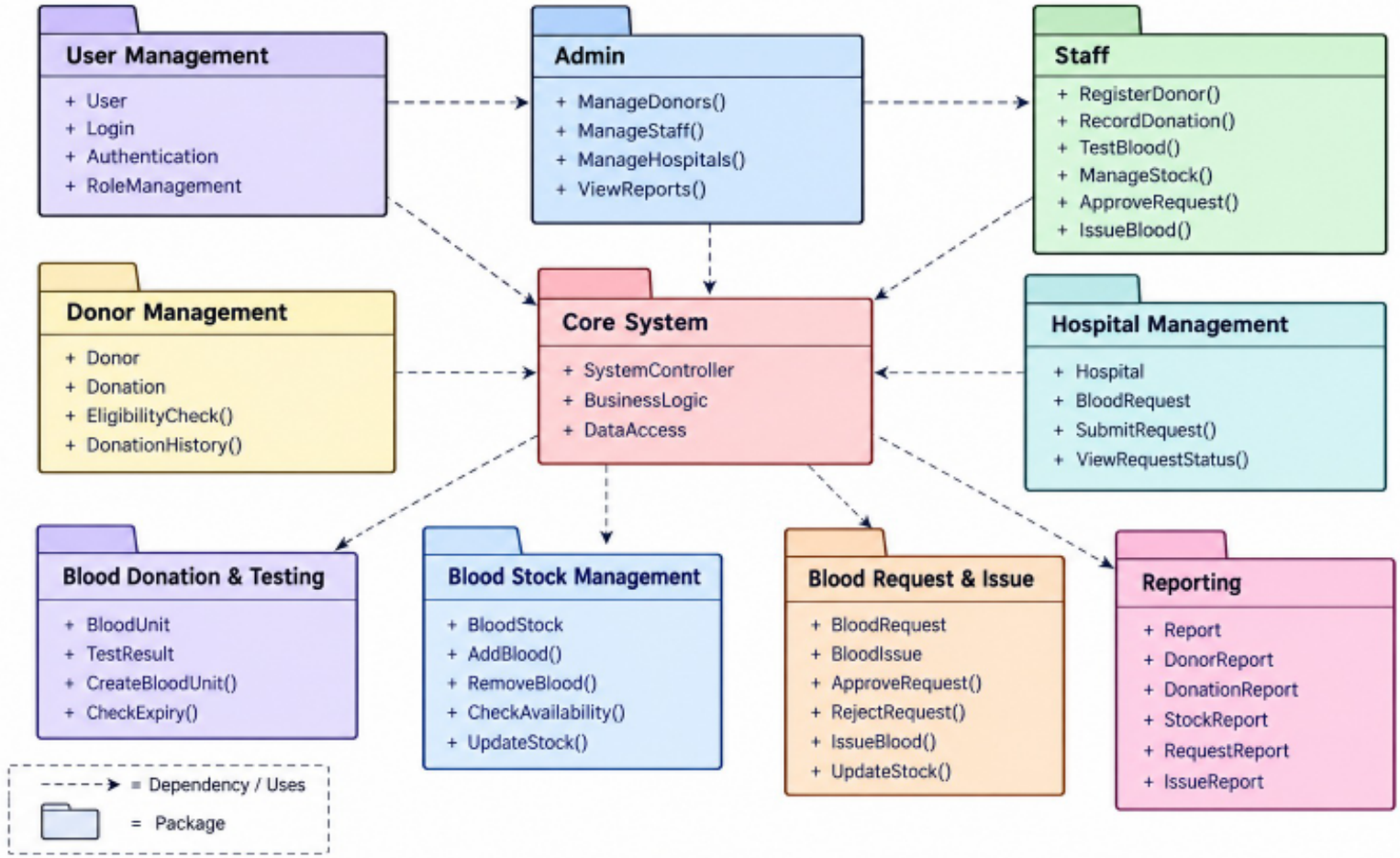

State Diagram – Blood Unit (Blood Bank Management System)
collect blood
Donor
donates blood
Collected
(Blood collected from donor)
entry / set collectionDate
do / generate unitId
send for testing
Testing
(Blood under test)
entry / set testStatus = Pending
do / perform tests
test completed
Test Passes?
No
Rejected
(Not fit for use)
entry / set status = Rejected
do / discard unit
dispose
Yes
Available
(Ready for storage)
entry / set status = Available
do / updateStock
add to inventory
In Stock
(Stored in blood bank)
entry / updateStock
do / checkExpiry
expiry date reached
Expired
(Not usable)
entry / set status = Expired
do / removeFromStock
discard
Hospital
request & receive blood
Issued
(Given to hospital)
entry / set status = Issued
do / updateStock
do / generateIssueRecord
approve request & issue blood
Allocated
(Reserved for request)
entry / set status = Allocated
do / upduceAvailableStock
check availability & approve
Pending Request
(Waiting for approval)
entry / set status = Pending
do / notifyStaff
reject request
Rejected Request
(Not approved)
entry / set status = Rejected
do / notifyHospital
send request
Requested
(Blood request created)
entry / set status = Requested
do / checkStock
submit blood request
Hospital
Legend:
Initial State
Final State
State
Decision
Transition
External Actor
Main States of Blood Unit
1. Collected – Blood collected from donor
2. Testing – Testing in progress
3. Available – Passed test and ready for storage
4. In Stock – Stored in blood bank
5. Allocated – Reserved for a hospital request
6. Issued – Given to hospital
7. Expired – Not usable, discarded
8. Rejected – Failed test / not fit for use

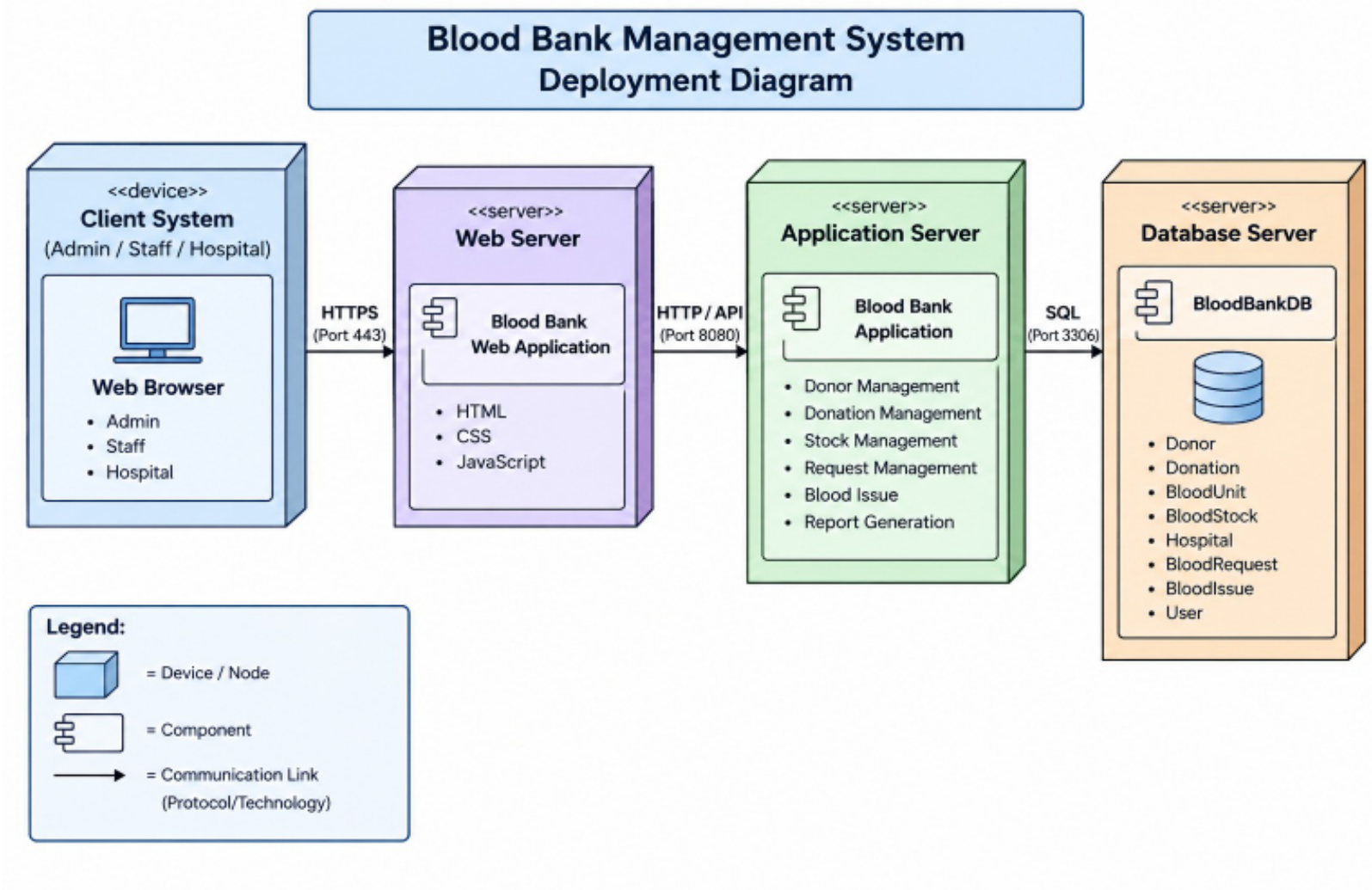

Activity Diagram – Blood Bank Management System
Donor
Staff
System
Hospital
Admin
Register as Donor
Provide Personal Details
Verify Donor Information
Check Eligibility
Eligible?
No
Inform Ineligible
Yes
Give Blood (Attend Donation Camp)
Record Donation Details
Create Blood Unit & Perform Tests
Test Passed?
No
Reject Blood Unit
Yes
Create Blood Unit
View Updated Stock
Update Blood Stock
Login to System
Check Blood Availability
Submit Blood Request (blood group & units)
Available?
No
Review Request & Check Stock
Send Request to Staff (for review)
Approve?
No
Reject Request
View Rejection Status
Yes
Reserve Blood Units
Update Stock
View Approval & Issue Status
Prepare Blood for Issue
Record Blood Issue
Receive Blood (Collect from Blood Bank)
Update Stock After Issue
Generate Issue Record
Generate Reports
View Reports (Donor / Donation / Stock / Request / Issue)
Legend:
= Start / End
= Activity
= Decision
= Flow
= Swimlane (Actor / Role)


Collaboration Diagram – Blood Bank Management System
«actor»
Donor
1: RegisterDonor()
2: Confirmation()
«boundary»
Donor Management
+ registerDonor()
+ checkEligibility()
3: createDonation()
4: donorDetails()
5: donateBlood()
«entity»
Donation
+ recordDonation()
+ createBloodUnit()
6: createBloodUnit()
7: testResult()
«entity»
BloodUnit
+ testBlood()
+ markAsAvailable()
8: addToStock()
9: updatedStock()
«entity»
BloodStock
+ addBlood()
+ checkAvailability()
«actor»
Hospital
10: submitRequest()
11: requestStatus()
«boundary»
Blood Request
+ createRequest()
+ approveRequest()
12: checkAvailability()
13: availabilityResult()
«control»
Staff
15: createIssue()
16: issueDetails()
14: approveRequest()
17: issueRecord()
«entity»
Blood Issue
+ issueBlood()
+ updateStock()
«actor»
Admin
19: generateReport()
20: reportData()
«entity»
Report
+ generateDonorReport()
+ generateStockReport()
Legend:
= Object / Component
= Actor
= Message / Communication
= System Boundary (not shown)


Component Diagram – Blood Bank Management System
<<component>>
User Interface
(Web / Desktop / Mobile)
+ Login
+ Dashboards
+ Forms
+ Reports View
HTTP/HTTPS
<<component>>
Application Layer
(Business Logic)
+ AuthenticationService
+ DonorService
+ DonationService
+ RequestService
+ ReportService
JDBC
<<component>>
Data Access Layer
(DAO / Repository)
+ UserDAO
+ DonorDAO
+ DonationDAO
+ RequestDAO
+ ReportDAO
JDBC
<<component>>
Database
(MySQL)
<<component>>
Donor Management
+ RegisterDonor()
+ UpdateDonor()
+ CheckEligibility()
+ ViewHistory()
<<component>>
Donation Management
+ RecordDonation()
+ TestBlood()
+ CreateBloodUnit()
+ UpdateStatus()
<<component>>
Blood Stock Management
+ AddBlood()
+ RemoveBlood()
+ CheckAvailability()
+ UpdateStock()
<<component>>
Blood Request Management
+ CreateRequest()
+ ApproveRequest()
+ RejectRequest()
+ ViewRequestStatus()
<<component>>
Blood Issue Management
+ IssueBlood()
+ UpdateStock()
+ GenerateIssueRecord()
<<component>>
Reporting
+ GenerateDonorReport()
+ GenerateDonationReport()
+ GenerateStockReport()
+ GenerateRequestReport()
+ GenerateIssueReport()
IDonorService
(provided interface)
IDonationService
(provided interface)
IBloodStockService
(provided interface)
IRequestService
(provided interface)
IIssueService
(provided interface)
IReportService
(provided interface)
<<component>>
Email Service
(SMTP)
Send Notifications
(e.g., request status, alerts)
<<component>>
SMS Gateway
Send SMS Alerts
(e.g., donation reminders)
<<component>>
File Storage
Store Reports / Documents
(PDF, Excel)
Legend
= Component
= Provided Interface
= Dependency
= Communication / Data Flow
= Port

Package Diagram – Blood Bank Management System




Blood Bank Management System
Deployment Diagram
<<device>>
Client System
(Admin / Staff / Hospital)
Web Browser
Admin
Staff
Hospital
HTTPS
(Port 443)
<<server>>
Web Server
Blood Bank
Web Application
HTML
CSS
JavaScript
HTTP / API
(Port 8080)
<<server>>
Application Server
Blood Bank
Application
Donor Management
Donation Management
Stock Management
Request Management
Blood Issue
Report Generation
SQL
(Port 3306)
<<server>>
Database Server
BloodBankDB
Donor
Donation
BloodUnit
BloodStock
Hospital
BloodRequest
BloodIssue
User
Legend:
= Device / Node
= Component
= Communication Link
(Protocol/Technology)